NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *February 2, 2026*

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

Adviser: Nuno Preguiça
Full Professor, NOVA University Lisbon

Co-adviser: Vítor Duarte
Assistant Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *February 2, 2026*

Abstract

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis LaTeX (novathesis) (LaTeX) template will respect them. Otherwise, the default rule in the novathesis template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more · The last keyword

Resumo

Independentemente da língua em que a dissertação está escrita, geralmente esta contém pelo menos dois resumos: um resumo na mesma língua do texto principal e outro resumo numa outra língua.

A ordem dos resumos varia de acordo com a escola. Se a sua escola tiver regulamentos específicos sobre a ordem dos resumos, o template (LaTeX) novathesis irá respeitá-los. Caso contrário, a regra padrão no template novathesis é ter em primeiro lugar o resumo *no mesmo idioma do texto principal* e depois o resumo *no outro idioma*. Por exemplo, se a dissertação for escrita em português, a ordem dos resumos será primeiro o português e depois o inglês, seguido do texto principal em português. Se a dissertação for escrita em inglês, a ordem dos resumos será primeiro em inglês e depois em português, seguida do texto principal em inglês. No entanto, esse pedido pode ser personalizado adicionando um dos seguintes ao arquivo `5_packages.tex`.

```
\abstractorder(<MAIN_LANG>):={<LANG_1>,...,<LANG_N>}
```

Por exemplo, para um documento escrito em Alemão com resumos em Alemão, Inglês e Italiano (por esta ordem), pode usar-se:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Relativamente ao seu conteúdo, os resumos não devem ultrapassar uma página e frequentemente tentam responder às seguintes questões (é imprescindível a adaptação às práticas habituais da sua área científica):

1. Qual é o problema?
2. Porque é que é um problema interessante/desafiante?
3. Qual é a proposta de abordagem/solução?
4. Quais são as consequências/resultados da solução proposta?

Palavras-chave: Primeira palavra-chave · Outra palavra-chave · Mais uma palavra-chave · A última palavra-chave

Contents

List of Figures

List of Tables

Acronyms

ACID Atomicity, Consistency, Isolation, Durability *(pp. 9, 11, 18)*
AI Artificial Intelligence *(pp. 4, 7, 12, 18, 19)*
API application programming interface *(pp. 7–9, 13, 27, 28)*

BaaS Backend-as-a-Service *(pp. v, 4, 7, 8, 18)*
BASE Basically Available, Soft State, Eventually Consistent *(pp. 9, 11)*
BPMN Business Process Model and Notation *(p. 4)*

CRUD Create, Read, Update, Delete *(pp. 8, 20)*
CSV Comma-Separated Values *(pp. 22, 24, 28)*

DI Department of Computer Science *(pp. 1, 3, 7, 10, 21)*

ECTS European Credit Transfer and Accumulation System *(p. 26)*
EJS Embedded JavaScript *(p. 9)*
ERD Entitiy-Relationships Diagrams *(p. 4)*

FCT NOVA School of Science and Technology *(pp. 1, 3, 7, 21)*

HTML HyperText Markup Language *(pp. 9, 14)*
HTTP Hypertext Transfer Protocol *(pp. 21, 27)*
HTTPS Hypertext Transfer Protocol Secure *(p. 27)*

I/O Input/Output *(p. 9)*
IDE integrated development environment *(p. 16)*
ISR Incremental Static Regeneration *(pp. 13, 19)*

JDBC Java Database Connectivity *(p. 14)*
JPA Java Persistence API *(p. 9)*

JSON JavaScript Object Notation *(pp. 10, 22, 28, 35)*
JSONB JavaScript Object Notation Binary *(pp. 10, 35)*
JSX JavaScriptXML *(p. 12)*

LLM large language model *(pp. 7, 15)*

MVP Minimum Viable Product *(p. 16)*

NoSQL Not only SQL *(pp. v, 4, 9, 11)*
NOVA NOVA University Lisbon *(pp. 1, 3, 7, 21)*
novathesis NOVAthesis LaTeX *(pp. i, iii)*

RBAC role-based access control *(p. 27)*
RLS Row-Level-Security *(pp. 8, 18, 19, 27, 35)*

SPA Single Page Application *(p. 11)*
SQL Structured Query Language *(pp. v, 4, 8–10)*
SSG Static Site Generation *(pp. 13, 19)*
SSL Secure Sockets Layer *(pp. 21, 25)*
SSR Server-Side-Rendering *(pp. 13, 19)*
STOMP Simple Text Oriented Messaging Protocol *(p. 14)*

TLS Transport Layer Security *(pp. 21, 27)*

UI User Interface *(pp. 12, 13, 15, 16, 19, 20, 22)*
UX User Experience *(pp. 2, 4, 11, 17)*

WYSIWYG What You See Is What You Get *(pp. 14, 22)*

1

Introduction

1.1 Context and Background

The transition of students from academic life to the professional market or advanced research is a key part of the university experience. This is mainly supported through three different distinct paths at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT):

- **Undergraduate Internships**: Primarily offered by external companies to provide students with their first professional experience.
- **Master's Dissertations**: Scientific thesis typically conducted within the university, focusing on academic research under the supervision of professors.
- **Engineering Projects**: Practical projects proposed by external companies or the Department of Informatics itself, allowing Master's students to apply advanced engineering principles to real-world projects.

Managing those processes involves a complex coordination of multiple stakeholders: students seeking opportunities aligned with their skills and interests, professors coordinating editions, supervising students and also proposing dissertation themes, and external companies providing practical challenges. Within this ecosystem, a reliable digital platform is essential to act as a central hub for communication, proposal submission, and student placement.

1.2 Problem Statement

At DI at NOVA FCT, the management of internships, dissertations and engineering projects involves a multi-stage lifecycle that includes proposal submission, clarification and approval processes, making proposals available to students, assigning projects to students, report/dissertation submission, and providing documentation to juries. Currently, the department relies on a legacy platform to support some of these activities, however this

system presents several critical issues that hinder the efficiency of the department.
The most significant limitation is that the current platform is not being utilized for the Master's degree processes - including both scientific dissertations and engineering projects. At the moment it is exclusively used to manage undergraduate internships, meaning that the more complex workflows associated with Master's students are handled through manual, or non-standardized methods. This creates a lack of centralized data an increases the administrative burden on coordinators, professors and the secretariat.
Furthermore, even within its current scope, the legacy system suffers from several functional and technological gaps:

- **Communication Gaps and Reliability**: The current method for "*Call for Proposals*" is manual, requiring coordinators to send individual emails to companies, advertising the opportunity send proposals, for each edition. There are no safeguards to ensure that these emails follow specific deliverability rules, often resulting in messages being flagged as spam. Additionally, the system lacks the flexibility to create and edit email templates, preventing coordinators from customizing communication for different editions or context.

- **Inefficient Feedback Loops**: Although companies can submit proposals, there is no integrated mechanism for coordinators to provide direct feedback on those proposals for clarification. This forces corrections to be handled outside the platform, whereas a centralized system would allow companies to correct and resubmit proposals based on coordinator comments.

- **Stale Company Data**: The existing repository contains a large amount of outdated information, including companies that no longer exist or have changed their contacts or other information. There is a lack of a "validation" status to ensure only active and verified companies can send proposals.

- **Manual Protocol Management**: For new companies that have never participated in previous editions, the process of signing the required protocols is still entirely manual. The platform should automate delivery and tracking of these legal documents as soon as a new company registers.

- **Outdated Technology and User Experience (UX)**: The platform is built on an obsolete technology stack that is hard to maintain, difficult to update, and exposes security risks. At the same time, the user interface is not intuitive for external stakeholders, making it difficult for companies to register, manage employee accounts, or monitor the status of their proposals in real time.

In conclusion, the necessity for a new platform arises from the need to unify all academic degrees under a single digital infrastructure, replacing manual tracking with a secure, automated system that ensures data integrity and seamless experience for all stakeholders.

1.3 Objectives

The primary objective of this dissertation is to design and develop a modernized web-based platform that unifies and automates the management and entire lifecycle of undergraduate internships, Master's dissertations, and engineering projects for DI at NOVA FCT. The projects aim to transition from a fragmented, manual system to a centralized digital ecosystem.
To achieve this goal, the following specific objectives have been defined for the initial version:

- **Process Unification**: Integrate the lifecycles of undergraduate internships and Master's degree pathways (both scientific and engineering-focused) into a single platform, replacing the current manual methods.
- **Workflow Automation**: Implement automated procedures for the "*Call for Proposals*", the validation of proposals, the student *seriation* (ranking) and the assignment of students to a proposal.
- **Enhanced Communication and Feedback**: Establish direct feedback loops within the platform, allowing coordinators to request clarifications on proposals and ensuring companies can resubmit corrected versions.
- **Document and Protocol Management**: Automate the delivery, tracking, and storage of legal protocols, student CVs, and recommendation letters.
- **Administrative Oversight**: Develop advanced administrative tools, including "*superuser*" access for student support, detailed system logs for auditing, and validation workflow for new company registrations.

1.4 Methodology

This project will follow an Agile/Scrum methodology, characterized by iterative development, frequent feedback loops [39] to ensure the final platform aligns with the university's complex requirements. The work plan is structured into several phases, prioritizing rapid prototyping followed by a transition to a high-performance custom architecture.
To achieve this goal, the following methodological steps will be used for the design, implementation, and validation of the system:

1. **Requirement Gathering and Analysis**: A thorough analysis of the legacy system and current manual processes was conducted to identify functional and non-functional requirements. This phase includes the definition of user roles (*Admin*, *Coordinator*, *Student*, *Professor*, *Company*, *Secretariat*) and their respective use cases.

2. **System Modeling and Design**: Utilizing Business Process Model and Notation (BPMN) to map complex workflows [27] and Entitiy-Relationships Diagrams (ERD) to design the database schema [7]. This phase also involves planning the system architecture.

3. **AI-Assisted Rapid Prototyping (*Vibe Coding*)**: The initial implementation phase will leverage *Vibe Coding* tools, such as Lovable, Figma Make or Google AI Studio, to quickly generate the frontend and core functional interfaces [9]. These tools excel at creating user interfaces through iterative natural language prompting. During this stage, Supabase will be utilized as Backend-as-a-Service (BaaS) to provide an immediate infrastructure for authentication, database management, and initial data structures via SQL[47]. The prototype produced in this phase will serve as the baseline that will be iteratively improved and hardened. This approach allows for the rapid fulfillment of most functional requirements while maintaining a live, testable version of the platform.

4. **Progressive Refinement and Continuous Development**: Once the prototype reaches a stable state and meets the core functional requirements, the focus shifts from "*getting this working*" to improving code quality, structure, and user experience. In this phase the existing codebase generated by AI is refactored, extended, and optimized manually and potentially still with the help of AI-assisted tools. The goal is to evolve the prototype into a maintainable system, improving UX, performance, and adherence to architectural guidelines.

5. **Custom Backend Evolution**: To support more complex business logic - such as specialized student *seriation* (ranking) algorithms and robust administrative auditing - the system progressively transitions towards a dedicated custom backend, likely using Spring Boot [41]. While Supabase remains valuable for rapid development, a Spring Boot-based architecture provides stronger domain modelling, type safety and maintainability through its layered structure (Controller, Service, Repository). This evolution builds on top of the existing data model and functionality, ensuring that the platform remains usable while gaining scalability and reliability for long-term university use.

6. **Verification and Validation**: The platform will be tested against the initial requirements, ensuring that automated emails follow specific deliverability rules avoiding being flagged as spam. Additionally, the seriation logic will be rigorously validated to ensure it correctly handles student placements and matching according to pre-defined criteria.

7. **Documentation and Evaluation**: There will be a continuous documentation of the architecture and design decisions, providing detailed justifications for technical choices such as the preference for SQL over NoSQL for structured academic data,

and other technical choices. At the end, the platform and development process will be tested and evaluated with respect to the initial goals, highlighting limitations and possible directions for future work.

2

State of the art and Technologies

This chapter examines the current state of software engineering practices and technologies relevant to building a modern platform for the DI at NOVA FCT. The goal is to overcome limitations of the legacy system - such as fragmented data, manual workflows, and inefficient communication - by evaluating contemporary approaches to backend architectures, data persistence, frontend frameworks, and AI-assisted development workflows. Particular attention is given to the emerging paradigm of AI-assisted software development, often described as "*Vibe Coding*" [14] which supports rapid prototyping by allowing developers to steer large language models (LLMs) through natural-language interactions instead of traditional line-by-line coding.

Modern web application stacks are characterized by a wide range of choices at each layer, from infrastructure and backend services to frontend frameworks and development workflows. On the backend, developers must choose between fully managed BaaS platforms [16] and custom backends that offer fine-grained control over domain logic, performance, and governance. On the frontend, single-page web applications and component-based frameworks (such as React, Angular, or Vue) are commonly combined with RESTful or GraphQL APIs, enabling decoupled client-server architectures and iterative delivery of new features.

While "*Vibe Coding*" and other AI-assisted practices promise faster experimentation and reduced boilerplate, they also introduce new trade-offs regarding maintainability, security, and auditability in production systems [38].

The remainder of this chapter critically reviews these technologies and architecture choices in light of the requirements identified for the new platform.

2.1 Backend Architectures and BaaS

The choice of backend architecture directly influences development speed, scalability, operational complexity, and the ability to encode complex business rules such as student seriation and multirole access control. For this project, two main paradigms are considered: Backend-as-a-Service solutions, which offer ready-made infrastructure and managed

services, and custom backends built with mature frameworks such as Spring Boot and Express.js, which provide full control over domain modeling, integration patterns, and deployment environments. [5]

BaaS platforms typically optimize for rapid time-to-market by abstracting away server provisioning, database management, and authentication, making them well-suited for early-stage prototyping and small teams [1]. In contrast, custom backends allow tailoring of data models, ownerships of the entire codebase, and fine-grained observability, which becomes critical for institutional applications that must satisfy compliance, long-term maintainability, and integration with existing academic processes.

Feature	**Supabase (BaaS)**	**Spring Boot**	**Node.js (Express)**
Architectural Pattern	Managed Infrastructure	Layered / Monolithic	Middleware-based
Development Velocity	Accelerated (Managed)	Moderate (Boilerplate)	High (Minimalist)
Domain Control	Ecosystem-dependent	High (Type-safe)	High (Flexible)
Integrations	Webhooks & Extensions	Extensive / Enterprise	Package-rich (NPM)
Security Model	Row-Level Security (RLS)	Spring Security (OIDC/JWT)	Custom Middleware

Table 2.1: Comparison of Backend Frameworks and Platforms

2.1.1 Supabase (Backend-as-a-Service)

Supabase is an open-source BaaS stack that builds on PostgreSQL and exposes automatically generated REST APIs through PostgREST [45], significantly reducing boilerplate CRUD implementation effort. By providing an integrated suite of tools - including database, authentication, storage, and real-time capabilities - Supabase enables rapid development cycles while leveraging a relational data model that is compatible with established SQL tooling and practices. [47]

A central feature of Supabase is its built-in authentication system, which supports user registration, password-based login, magic links, and role-based access control[42], simplifying the enforcement of different permission profiles for students, companies, professors and administrators. Row-Level-Security (RLS) policies can be defined directly at the database level, ensuring that authorization rules are enforced close to the data and cannot be bypassed by misconfigured API clients [46]. For logic that goes beyond standard CRUD operations - such as orchestrating notification workflows or managing student seriation - Supabase offers serverless “Edge Functions”, typically written in TypeScript or JavaScript, which run in a managed environment near end users to minimize latency. [43]

2.1.2 Spring Boot (Custom Backend)

Spring Boot is a widely adopted framework for building production-grade Java or Kotlin backends and is especially suitable for complex, domain-driven systems that require strict typing, layered architectures, and extensive integration capabilities. Its opinionated configuration model and extensive ecosystem (Spring Data, Spring Security, Spring Cloud) allows developers to structure applications into controllers, services, and repositories, facilitating clear separation of concerns and maintainable domain models. [41]

For academic workflows that must support student ranking algorithms, complex eligibility rules, and auditable decision-making processes, the type safety and testability provided by Spring Boot are particularly advantageous. The framework integrates with JPA and tools like Hibernate Envers to support automatic auditing of entity changes [41, 6], which helps track user actions, state transitions, and login-related events - capabilities that are often required for administrative oversight and internal compliance in university environments.

2.1.3 Node.js and Express (Custom Backend)

Express.js is a minimalist web framework for Node.js that offers a lightweight and flexible foundation for building HTTP APIs using JavaScript or TypeScript [13]. Its event-driven, non-blocking I/O model allows a single process to handle many concurrent connections efficiently, making it well-suited for notification-heavy workloads [26].

The Node.js ecosystem provides a rich set of libraries for templated email generation and delivery, including tools such as Handlebars or EJS to render dynamic HTML email bodies [17, 12] for coordinators and administrators. Combined with tasks schedulers or message queues, Express-based backends can orchestrate asynchronous communications and background jobs [26], helping address a key functional gap of legacy systems that rely on manual email workflows and ad hoc templates.

2.2 Data Persistence: SQL vs. NoSQL

The selection of a data persistence model is a fundamental architectural decision that dictates how the system ensures data integrity, scalability, and retrieval efficiency [52, 4]. Given the complex ecosystem of the thesis and internships management, which involves multiple stakeholders (Students, Professors, Companies, Secretariat, etc...) and multi-stage lifecycle workflows for academic projects, this section evaluates the trade-offs between Relational (SQL) and Non-Relational (NoSQL) databases.

The choice between SQL and NoSQL has complex implications for institutional systems. Relational databases prioritize data integrity and consistency through Atomicity, Consistency, Isolation, Durability (ACID) compliance [34], while Non-Relational systems typically follow the Basically Available, Soft State, Eventually Consistent (BASE) model, trading immediate consistency for horizontal scalability and schema flexibility [51].

Criterion	Relational (SQL)	Non-Relational (NoSQL)
Schema Definition	Rigid/Predefined	Schema-less/Dynamic
Consistency Model	ACID Compliance	BASE (Eventual)
Data Integrity	Enforced (Foreign Keys)	Application-level logic
Query Complexity	Native Relational Joins	Aggregation Pipelines
Scaling Strategy	Vertical Scaling	Horizontal Partitioning
Institutional Fit	High (Structured Records)	Low (Consistency Risks)

Table 2.2: Comparison of Data Persistence Models

2.2.1 The Relational Model (SQL)

Relational databases structure data into tables with predefined schemas, utilizing Foreign Keys to establish strict relationships between entities [18]. This approach provides fundamental guarantees about data quality and consistency that are essential for institutional systems.

- Structured Data and Integrity: The academic context requires rigorous data consistency. For instance, a Candidacy must strictly link to an existing Student and a valid Proposal. The relational model enforces referential integrity through primary and foreign key constraints, preventing "orphaned" records - a situation where a child record references a non-existent parent record [40]. This is particularly critical in the DI management platform, where deleted Companies should not retain active Proposals, and Students cannot be assigned to non-existent projects.

- Referential integrity mechanism ensure that every foreign key in a child table corresponds to a valid primary key in the parent table, maintaining logical consistency across the entire database. Beyond preventing data anomalies, referential integrity supports cascading updates and deletions, ensuring that changes propagate correctly across related entities [18, 40].

- Complex Querying: The platform requires complex join operations to generate views - for example, listing all proposals for a specific edition filtered by a company's validation status, or retrieving all students that are candidates to a particular internship. SQL excels at these complex relationships through its mature and optimized query language, which can efficiently traverse multiple table relationships in a single query [23].

- PostgreSQL: This project will initially utilize PostgreSQL (via Supabase for rapid prototyping). Beyond standard SQL features, PostgreSQL offers support for complex data types (JSON/JSONB, arrays, enums) and Stored Procedures [28, 32]. This allows

the system to handle semi-structured data - such as like flexible feedback forms or email template configurations - while maintaining the benefits of a relational schema for academic entities.

2.2.2 The Non-Relational Model (NoSQL)

NoSQL databases (e.g., document stores like MongoDB or key-value systems like Redis) offer schema flexibility and horizontal scalability, often at the cost of immediate consistency (ACID properties) [52]

- Flexibility vs Structure: While NoSQL allows for rapid iteration of data models without schema migrations, the academic domain is highly structured. Entities such as Users, Editions, Proposals, Companies and Students have well-defined attributes that rarely change dynamically or unexpectedly. The governance requirements of an academic institution demand clear, predictable data structures rather than evolving, loosely-typed records.

- Horizontal Scalability: A key advantage of many NoSQL systems is the ability to easily scale horizontally across multiple nodes, distributing data and load to handle very high traffic volumes or massive datasets. This is attractive for large, globally distributed consumer applications, but less critical for an institutional platform whose workload is bounded by the number of students, companies and editions. In this context, the added operational complexity of managing a distributed NoSQL cluster is not justified by the expected scale, especially when a single relational database can already handle the anticipated load with proper indexing and tuning.

- Consistency Challenges: In workflows like Student Seriation, data consistency is essential. A NoSQL approach, which often relies on eventual consistency, could introduce race conditions where a student is assigned to a project that is no longer available or where multiple simultaneous requests lead to conflicting state changes. The BASE model prioritizes availability and partition tolerance over strict consistency, which is acceptable for read-heavy analytics systems but unsuitable for transactional workloads that require immediate correctness.

2.3 Frontend Frameworks and User Experience

The "Outdated User Experience (UX)" of the legacy system is identified as a critical failure point, particularly for external stakeholders such as companies who struggle to register, manage accounts, or monitor proposal status. To address these limitations, the new platform must transition from static, server-rendered pages to a modern, reacting Single Page Application (SPA) or hybrid architecture that enables faster navigation, richer interactivity [35], and consistent interfaces across distinct user roles.

2.3.1 React.js

React is a free, open-source JavaScript library maintained by Meta that specializes in building interactive user interfaces through component-based architecture [35, 36]. Unlike traditional web development where the entire page must be reloaded on data changes, React efficiently re-renders only the components that have actually changed, using its virtual DOM reconciliation process to minimize performance overhead [24]. React uses JavaScriptXML (JSX), a syntax extension that allows developers to write HTML-like code directly within JavaScript [35], making the UI logic more readable and maintainable.

React is selected as the core library for building the platform's user interface for two primary reasons:

1. Component-Based Architecture: React allows for the creation of reusable, self-contained UI components (e.g. "ProposalCards" where all the details from a proposal are, that can be used in multiple pages) [33]. This modular approach ensures consistency across the distinct interfaces required for Students, Professors, and Companies, while enabling efficient code reuse and simplified maintenance.

2. Synergy with AI-Assisted Development: The project methodology relies on Vibe Coding tools for rapid prototyping. Tools such as Lovable, Bolt.new, Figma Make and Google AI Studio are optimized to generate code specifically for the React ecosystem. Using React ensures that the code generated during the prototyping phase is immediately usable and extensible, eliminating the need for time-consuming library migrations.

2.3.2 Angular

Angular is a full-featured, opinionated frontend framework maintained by Google that provides an integrated solution for components, routing, forms, and state management in a single toolkit. Unlike React, which focuses on the view layer and relies on additional libraries for concerns such as routing, Angular ships with a more prescriptive structure (modules, services, dependency injection), which can be advantageous for large teams that value strong convetions.

Angular was considered as a viable alternative because if offers:

- Strong typing when used with TypeScript by default
- Built-in patterns for modularization and dependency injection
- Mature tooling for forms, validation, and reactive programming

However, given the project's emphasis on AI-assisted prototyping and the tight integration between *Vibe Coding* tools and the React ecosystem, React was preferred. Angular remains a valid choice for institutional systems, but in this context it would add friction to AI-assisted workflows and reduce reuse of generated components.

2.3.3 Next.js

Next.js is a full-stack React framework created and maintained by Vercel that extends React's capabilities [25] by providing built-in solutions for routing, data fetching, server-side rendering, and infrastructure - all without complex configuration. While React is a library focused solely on the UI layer and requires additional libraries for routing and server-side concerns, Next.js provides an integrated toolkit that handles both frontend rendering and backend API logic in a single project [25]. Next.js supports multiple rendering strategies: Server-Side-Rendering (SSR) for dynamic content rendered on-demand, Static Site Generation (SSG) for pre-rendering at build time, and Incremental Static Regeneration (ISR) for updating static content without full rebuilds [22].

Next.js is selected to complement React because it provides essential production-grade features:

- Routing and Performance: Next.js offers a file-system based router (App Router) [3] and rendering strategies such as SSR and SSG. SSR renders pages on demand for dynamic content, while SSG pre-renders static pages at build time, reducing server load times and enabling fast initial page loads [22], which is crucial for retaining the engagement of users. This flexibility allows different sections of the platform to use the optimal rendering strategy based on content volatility.

- Scalability and Full-Stack Capabilities: As the platform transitions from a prototype to a long-term solution, Next.js provides the architectural structure needed to manage complex API integrations, and full-stack workflows.

2.3.4 Real-Time Feedback and Interactivity

The legacy system suffers from "Inefficient Feedback Loops" where users must constantly refresh the web page or wait for emails to know the status of a proposal.

- Supabase Realtime (Prototyping Phase): During rapid prototyping with Supabase, Real-time subscriptions powered by WebSockets, allow the frontend to listen to database changes instantly and update the UI components without requiring page refreshes [44]. For example, a student receives an immediate "Selected" when a company confirms an interview result, this improves user engagement.

- Production Architecture: PostgreSQL LISTEN/NOTIFY with Spring Boot WebSocket: When transitioning to Spring Boot, real-time updates leverage PostgreSQL's native LISTEN/NOTIFY mechanism - a lightweight, built-in pub/sub system [31, 30] that requires no external message brokers. The workflow is straightforward.

    1. Database Trigger: When a Proposal status changes (e.g., from "Pending" to "Accepted") a PostgreSQL trigger automatically fires and sends a notification via `pg_notify()` to a named channel [31, 30].

2. Spring Boot Listener: A background thread in Spring Boot continuously listens on the PostgreSQL notification channel using JDBC's PostgreSQL driver extensions [19]: When a notification is received, it triggers an event in the application.
3. WebSocket Broadcast: Spring Boot WebSocket infrastructure (optionally using Simple Text Oriented Messaging Protocol (STOMP) for advanced message routing) [49, 41] broadcasts the update to all connected frontend clients subscribed to relevant topics. For instance, all students viewing proposals from a specific company receive live updates when that company changes a proposal's status.

2.3.5 Communication Experience (Email and In-App Notifications)

A major pain point of the legacy platform is the lack of structured, reliable communication. Coordinators have limited control over the deliverability of the emails, they are frequently flagged as spam, and users receive little feedback inside the system when proposals statuses or candidacies change.

To address this, the new platform combines configurable email templates with in-app notifications, ensuring that critical events are communicated both inside and outside the application.

- Template Management: The frontend will feature a What You See Is What You Get (WYSIWYG) markdown or HTML editor [29] (integrated into the Admin Dashboard) allowing coordinators to visually edit email templates, without the knowledge of those markup languages. This eliminates the need for manual template coding and reduces errors.

- Deliverability: To ensure emails reach users inboxes, the system architecture must integrate with an established transaction email service rather than relying on the error-prone, manual email workflows of the legacy system [21]. These services provide authentication protocols, monitoring, and detailed analytics to maintain sender reputation and maximize inbox placement [10].

- In-App Notifications: In parallel with email, the platform will generate in-app notifications for key events along the proposal and candidacy lifecycle, such as "clarification requested", "proposal validated", "seriation completed". Users see these notifications immediately when logged in, and they are linked to the relevant screen (proposal detail, candidacy view), improving the overall interaction flow and making feedback loops more transparent.

Together, these mechanisms provide a coherent communication experience that replaces manual, unreliable emailing with configurable, trackable messages and immediate feedback inside the platform.

Feature	React.js	Angular	Next.js
Core Paradigm	Component-based Library	Opinionated Framework	Meta-framework (Full-stack)
State Management	External (Redux)	Built-in (Services/RxJS)	Integrated (Server/Client)
Rendering Strategy	Client-Side (CSR)	Client-Side (CSR)	Hybrid (SSR, SSG, ISR)
Typed Support	Optional (TypeScript)	Native (TypeScript)	Native (TypeScript)
Prototyping Synergy	High (Vibe Coding tools)	Moderate	High (Deployment-ready)

Table 2.3: Comparison of Frontend Technologies

2.4 AI-Assisted Development (Vibe Coding)

Traditional software development involves writing code manually, line by line. However, a new paradigm known as "Vibe Coding" has emerged, leveraging large language models (LLMs) to generate full-stack applications or complex User Interface (UI) components through natural language prompting [8, 38]. This section analyzes the capabilities of these tools to determine their viability for the rapid prototyping phase of the dissertation.

2.4.1 Concept and Capabilities

Vibe Coding tools differ from standard code assistants (like GitHub Copilot) by focusing on generating entire applications or functional modules rather than just autocomplete syntax. They typically integrate a development environment, a live preview window, and an AI agent capable of iterating on code based on user feedback. The primary advantage of this approach is the drastic reduction in time required to move from a conceptual requirement to a testable, interactive interface.

2.4.2 Analysis of Vibe Coding Tools

For this dissertation, four representative *Vibe Coding* tools were considered and explored: **Lovable, Bolt.new,** and **Figma Make**. They differ in scope (full-stack vs. frontend-centric), integration with existing workflows, and the amount of control they give developers over the generated code.

Criterion	Lovable	Bolt.new	Figma Make
Primary focus	End-to-end web apps	Full-stack code generation	Design-first interactive prototypes
Typical stack / output	React + Supabase	React, Angular, Vue, React Native, etc.	React + Supabase
Backend / DB support	Supabase backend & DB	Supabase backend & DB	Supabase backend & DB
Git / integration	Bi-directional GitHub sync	Bi-directional GitHub sync	One direction GitHub sync
Strengths	Complete environment, scaffolding, debugging, team-friendly	Framework flexibility, closer to “real” code	High visual fidelity, great for designers
Limitations	Mainly web/React; still evolving for large, complex domains	Rougher UX, more bugs and manual debugging	Generated code hard to reuse/maintain; more bugs; less suited to production

Table 2.4: Comparison of Vibe Coding Tools

Lovable

focuses on providing an end-to-end environment for building web applications with minimal friction. From a single interface, it can generate React-based UIs, configure Supabase-backed database and backend, and deploy the resulting application. When requirements are underspecified, Lovable scaffolds the missing pieces on both frontend and backend, and offers automatic debugging and safety checks that make it forgiving for users with limited programming experience. A distinctive advantage is bidirectional GitHub synchronization: projects can be edited in an external IDEs and then synced back to Lovable. Although it started as a prototyping tool, its feature set and collaboration capabilities make it a suitable for production-adjacent MVPs, internal tools and small applications [2].

Bolt.new

targets developers who want "prompt-to-full-stack" generation while retaining framework flexibility. It can scaffold applications across multiple stacks (React, Angular, Vue, React

Native, etc.) and is backed by Supabase for backend and database support, similar to Lovable. Compared to Lovable, Bolt places more emphasis on exposing the underlying code and project structure, which provides greater control but also demands more debugging and configuration effort. It also supports working with real repositories and two-way GitHub sync. For teams that must target specific frameworks (for example, Angular or React Native), Bolt is a more natural fit than tools that assume React only, but its UX is less polished for non-experts [2].

Figma Make

is tightly integrated into the Figma ecosystem and is therefore particularly attractive to designers. It starts from high-fidelity UI layouts and turns them into interactive prototypes that closely match the original design. This design-first approach is powerful for validating user flows and visual details with stakeholders. However, the generated code is harder to reuse outside Figma, and complexity grows quickly once custom logic and structure are layered on top. Although Figma Make supports backend integration (e.g., via Supabase), its current strength lies in rich prototypes rather than production-ready application code. For teams already working with Figma, its cost overhead is low, but they must be prepared to deal with frequent minor bugs and inconsistencies [2].

2.4.3 Limitations and Architectural Implications

Although *Vibe Coding* tools are powerful accelerators, they also raise several architectural concerns for an institutional platform.

First, they optimize for speed over structure. Generated code often prioritizes quick results rather than long-term maintainability, with oversize components and domain logic mixed into UI code. For a system that must be maintained across many years, this requires a deliberate refactoring step to align the code with a layered architecture (presentation, business logic, data access).

Second, these tools assume opinionated defaults for stack and infrastructure (for example, React + Supabase in Lovable, or specific project layouts in Bolt), which may conflict with institutional choices such as Spring Boot backend or stricter security and auditing policies. In this dissertation, Vibe-generated code is treated as a starting point that is progressively integrated into a more controlled environment, not as the final deployment artifact.

Third, there are governance, security and vendor-lock-in risks. Code generation and debugging typically run in external cloud environments with their own access and retention policies, and tight integrations (e.g., Figma Make + Figma) can couple parts of the system to specific providers. These risks are mitigated by limiting sensitive data in external tools and encapsulating AI-related features behind backend interfaces so that underlying providers can be swapped without rewriting the application.

Lastly, *Vibe Coding* does not remove the need for engineering skills and debugging. Generated applications may appear correct until they hit edge cases or scale, at which point conventional testing, tracing and refactoring are still required. Consequently, AI-assisted generation is used here to explore and accelerate, while critical data models, security controls and business logic are consolidated in a stable, hand-curated codebase that remains under developer control.

2.5 Summary and Justification of Technological Choices

The preceding sections surveyed alternative technologies for the backend, data persistence layer, frontend frameworks, and AI-assisted development. This subsection summarizes and justifies the concrete choices adopted for the platform, in light of the requirements and constraints identified in Chapter 3.

2.5.1 Backend Architecture

Section 2.1 compared BaaS solutions with custom backend built with mature frameworks such as Spring Boot and Express.js. BaaS platforms like Supabase are particularly attractive for early-stage projects because they provide read-made infrastructure for authentication, storage, and real-time APIs, enabling rapid prototyping with small teams.

For this project, Supabase is used in the initial phase to support AI-assisted, Vibe Coding-driven prototyping of the core workflows. However, the long-term target architecture is a custom backend implemented with Spring Boot. This choice is justified by:

- the need to code complex domain logic (student seriation rules, audit policies) with strong typing and explicit layering;
- the importance of testability and maintainability for an institutional system that is expected to evolve over multiple editions;
- the need to integrate cleanly with PostgreSQL features such as RLS and LISTEN/NOTIFY

In practice, the prototype built on Supabase is evolved rather than discarded: the data model and API contracts defined during prototyping are reused by the Spring Boot backend, minimizing migration effort.

2.5.2 Data Persistence

Section 2.2 contrasted relational (SQL) and non-relational (NoSQL) database models. Relational database prioritize integrity and consistency through ACID guarantees and referential constraints, while NoSQL systems offer schema flexibility and horizontal scalability at the cost of weaker consistency.

Given the structured nature of the problem domain-editions, students, companies, proposals, candidacies, and protocols - and the critical importance of correctness in operations such as seriation and assignments, a relational model is more appropriate.
The expected scale (bounded by the number of students and companies in the department) does not require the extreme horizontal scalability that motivates most NoSQL deployments.

PostgreSQL is therefore adopted as the primary data store because it:

- enforces referential integrity between core entities, preventing impossible states such as assignments to non-existent proposals or inactive companies ;
- supports RLS, which is essential for enforcing multi-tenant access rules directly at the database layer;
- provides JSON/JSONB columns for semi-structured data (*e.g.*, email templates, feedback metadata), allowing limited schema flexibility without introducing a separate NoSQL system.

2.5.3 Frontend Frameworks and Styling

Section 2.3 discussed React and Angular as representative frontend technologies. Angular offers an opinionated, full-stack framework, while React focuses on the view layer and relies on external libraries for routing and state. In this project React is chosen as the primary UI technology for two reasons:

- its component-based model fits well with the platform's need to offer multiple role-specific dashboards built from reusable components;
- most *Vibe Coding* tools use (Lovable, Bolt.new, Figma Make, Google AI Studio) target React as their default output, allowing AI-generated prototypes to be refined instead of rewritten.

Next.js is adopted as the framework around React because it simplifies routing and provides flexible rendering strategies (SSR, SSG, ISR) that can be matched to different views - for example, static generation for informational pages and server-side rendering for dashboards that require up-to-date data.

For styling, the platform can use a utility-first framework such as Tailwind CSS [37], which many AI coding assistants already target, enabling rapid conversion form designs to responsive layouts. However, the architecture does not depend on Tailwind specifically:

- it can be combined with or replaced by other styling approaches (CSS modules, design systems). The key requirement is that styling remains modular and consistent with the component-based frontend.

2.5.4 AI-Assisted Development (Vibe Coding)

Section 2.4 introduced Vibe Coding tools such as Lovable, Bolt.new and Figma Make. These tools are not part of the runtime architecture of the platform, but they strongly influence the development workflow.

For this project, AI-assisted tools are used in a complementary way:

- A tool like Lovable or Bolt.new can bootstrap full-stack React/Supabase applications and generate much of the initial UI and CRUD logic for core workflows.
- Figma Make can be used selectively to convert high-fidelity Figma designs into React components for visually complex views, which are then integrated into the main codebase.

However, these tools are treated as accelerators for prototyping, not as the final source of truth for the production system. The architectural implications are:

- Generated code is used as a starting point and then refactored into the layered architecture, ensuring clean separation between presentation, business logic and data access.
- Critical concerns - data model, security, seriation logic, auditing are implemented and reviewed in the main codebase (Spring Boot + PostgreSQL, React/Next.js) rather than being left hidden inside tool-specific projects.

In summary, Vibe Coding tools are justified as a way to reduce time-to-prototype and explore design alternatives while still converging on a conventional, maintainable architecture for the long-term institutional platform.

3

Requirements and Current System Analysis

3.1 Analysis of the Legacy System

The current internship management platform at DI at NOVA FCT is a web-based solution built on a deprecated technology stack. Although it supports the undergraduate internship life cycle, it presents critical limitations regarding security, architecture, and functionality. The section provides a systematic analysis of the current system's components and deficiencies, informed by direct exploration of the platform.

3.1.1 Infrastructure and Tech Debt

The system is hosted on a virtual machine and operates on an obsolete version of PHP dating back to approximately 2010, but since the platform was made even before that maybe there are some modules that are way older. This legacy codebase entails high maintenance risks and incompatibilities with modern libraries. Specifically, older PHP versions no longer receive security updates, leaving the platform vulnerable to known exploits that malicious actors routinely target [11].

A more critical vulnerability lies in the platform's network architecture: the system communicates exclusively over HTTP (without SSL/TLS encryption), representing a severe risk for transmitting sensitive student data and access credentials. Data transmitted over unencrypted HTTP is exposed to interception and eavesdropping, allowing attackers to read login credentials, personal information, and institutional data in plaintext [20].

Email Deliverability Crisis: A particularly severe technical deficiency resides in the email infrastructure. Due to the lack of modern authentication and deliverability configurations, transactional emails sent by the platform - intended for registration notifications, deadline reminders, call for proposals - are systematically flagged as spam by email providers [10]. This breakdown in communication disrupts the workflow between the department, students, and companies, reducing the operational effectiveness of the platform regardless of other functionality.

3.1.2 Functionalities and Limitations per Role

Administration and Coordination: The administrator profile provides a global overview of entities and proposals, allowing actions such as bulk student imports via CSV and basic permission management. A "*superuser*" feature exists, allowing administrators to access any user account without credentials for troubleshooting purposes. However, a critical gap exists in audit logging: while the system logs normal and superuser's logins and logouts, it lacks comprehensive action logging that tracks what users do during their sessions. For compliance and accountability, the platform should record specific administrative actions such as proposal status changes, student data modifications, permissions adjustments, and bulk import operations.

Content management relies on outdated, inflexible in-browser WYSIWYG editors for maintaining FAQs, Regulations, and Edition-specific information. Mass communication is rigid: the system offers no configurable templates for different stages of the workflow, relying instead on a single standard message for inviting companies.

Proposal Management and the Master thesis Gap: The platform includes interfaces for companies or professors to submit internship proposals and Master's dissertation themes. However, these dissertation theme submission and management workflows are completely inoperative - they exist as "dead data" in the system, forcing the department to manage dissertation assignments, publication and monitoring via external tools, negating the value of having collected this data in the platform.

For undergraduate internships, the "Project Classification" workflow allows coordinators to accept, reject, or request clarifications by annotating proposals. However, interaction remains limited: coordinators can toggle whether a proposal awaits company response, but the feedback mechanism lacks fluid integration for iterative proposal refinement.

Students and Companies: The student interface allows ranking of internship preferences and the consultation of imported academic records. For companies, human resources management is inefficient: there is no clear hierarchy or efficient mechanisms for managing multiple employees within a single company entity. The overall UI is outdated, hindering navigation and making it difficult for users to perceive current application status.

3.1.3 Data and Reports

The current system suffers from a critical lack of interoperability. There are no native features to export complex data (student lists, proposals, results of attributions) into open, standard format such as CSV or JSON, which complicates operations that the coordinators might need to make with the data outside the platform. Furthermore, company information is often outdated, as there are no periodic revalidation mechanisms to refresh company profiles and contact details ("Stale Company Data").

3.2 Requirements Gathering

Following the systematic analysis of the legacy platform and the identification of its critical failures, a comprehensive set of requirements was elicited for the new integrated platform. The elicitation process focused on addressing the needs of the diverse stakeholders involved in the academic lifecycle (students, companies, coordinators, secretariat, and administrators), with the goal of transitioning from fragmented spreadsheet-based management to a unified digital ecosystem supporting both undergraduate internships and master's dissertations. The resulting requirements were prioritized and refined in collaboration with the thesis advisers to ensure alignment with institutional constraints regarding data protection, interoperability, and operation feasibility.

3.2.1 Elicitation Methods

To ensure the new platform addresses real operational pain points and does not introduce new barriers to existing workflows, requirements were gathered through a multi-faceted approach:

- **Legacy System Analysis**: Direct exploration and reverse-engineering of the existing platform to identify functional gaps (e.g. non-functional Master's modules) and technical debt (e.g. email deliverability issues). This analysis, presented in Section 3.1, served as the foundation for identifying core functional requirements.

- **Document Analysis**: Review of current regulations, current internship rules, and manual processes currently used to manage Master's dissertations and feedback tracking. This provided insight into established business logic that must be preserved and systematized in the new platform.

- **Workflow Modeling**: Mapping of the "*as-is*" manual processes and system workflows against desired "*to-be*" automated and digitized workflow. Specifically for proposal submission and validation, student seriation algorithms, registrations workflows, company hierarchies, and feedback management. Process models were documented to clarify state transitions, decision points, and inter-stakeholder communication patterns.

- **Stakeholder Consultations**: Informal discussions with thesis advisers and the current coordinator responsible for the platform administration provided critical insights into administrative pain points, compliance requirements, and feature priorities. These discussions informed the prioritization of features and validated the proposed solution direction.

3.2.2 Stakeholders

The system must support multiple distinct user groups. However, the role hierarchy reflects the institutional structure where coordinators are designed professors with expanded administrative responsibilities, rather than a separate administrative role.

- **Students**: Seek to consult and review available internship and dissertation/project proposals within their enrolled edition, rank proposals according to their preferences, track the status of their candidacies and assignments, access their assigned supervisor's contact information, and, if required by the coordinators, submit documentation (CV, motivation letter, or recommendations) for specific proposals.
- **Company Representatives**: Aim to register company information and manage organizational hierarchy (parent companies and sub-entities or departments), submit internship and Master's dissertations proposals during open call periods, receive and respond to feedback from coordinators regarding proposal clarifications, manage company employees and designate contacts responsible for specific proposals, schedule and present their proposals during coordinator provided time slots, and track interview progress and student candidacies through a dashboard.
- **Professors**: Intent to propose scientific dissertation themes for the Master's degree, supervise students assigned to their dissertation topics, provide feedback on dissertation progress.
- **Edition Coordinators (Professor variant, Head Coordinator)**: Is a professor with comprehensive administrative responsibilities for the system. Their responsibilities include configuring and managing editions (setting edition type - undergraduate internship or Master's dissertations, defining proposals' deadlines, configuring available time slots for company presentations, creating edition-specific information pages such as FAQs and contact pages), validate submitted proposals through a feedback-and-clarification workflow, control the attributions process (being able to manually add a student to an internship outside seriation), generating and customizing email templates for all communications stages, publish news and announcements targeting different user groups, managing user pre-registration and bulk imports via CSV, and overseeing the finalization of internship/dissertation protocols. The head coordinator also maintains comprehensive audit logs and manages permissions assignments for sub-coordinators and other edition-specific administrative tasks.
- **Sub-Coordinators (Professor variant)**: Professors designed by the head coordinator to assist with proposals validation and feedback workflows for specific edition, but without access to system-wide configuration. They inherit permissions from the Professor's role but with additional privileges limited to proposal evaluation and validation, configuration of edition-specific resources (FAQs, contact pages, email templates) within their assigned edition.

- **Secretariat/Administrative Staff**: Manage financial and legal formalization of internships and dissertations, specifically verifying payments from companies for protocol signing and managing storage and archival of signed agreements. They require a view-only access to status dashboards showing active internships and their completion status.

- **Administrators**: Handle the infrastructure-level operations that fall outside the coordinator's scope, such as system backups, database maintenance, server updates, monitoring system health, managing SSL certificates and security configurations, responding to critical technical incidents, and consulting detailed audit logs for technical troubleshooting and compliance verification. They do not manage user accounts, permissions, or edition-specific configurations - those remain the responsibility of the head coordinator. This role is typically filled by dedicated IT staff rather than a professor.

3.2.3 Functional Requirements

The functional requirements are categorized by feature set to ensure modular development, testing and clear traceability of stakeholder needs. Detailed specification for all functional requirements are presented in Annex I, Tables I.1-I.7. The following subsections provide a summary of each category:

FR-1x: Edition and Configuration Management: This category covers the administrative features coordinators need to create and manage academic editions. It includes creating editions for undergraduate internships or Master's dissertations, configuring key timelines (proposal submission, interviews, finalization), and managing edition-specific content such as FAQs, regulations and contact information. Coordinators can also schedule company presentation slots and publish targeted announcements using reusable email templates, replacing the legacy platform's rigid configuration mechanisms and enabling each edition to be tailored to its specific needs.

FR-2x: Proposal Submission and Classification: This category groups the workflows through which companies and professors submit and refine internship or dissertation proposals. The system supports detailed proposal submission, iterative clarification and feedback (rather than simple accept/reject), and final Classification by coordinators. Companies can reuse proposals from previous editions, adapt them to new contexts, and schedule oral presentations in coordinator-defined time slots. Automated notifications keep proposers informed about submission deadlines and feedback windows.

FR-3x: Student Candidacies and Seriation: This category automates the process by which students apply to proposals and are ranked for assignment. Students can browse available proposals for their edition, rank a limited number of preferences, and,

when required, attach supporting documents such as CVs or motivation letters. The system executes a seriation algorithm based on predefined criteria (*e.g.*, ECTS, average grade, coordinator-defined weights) to produce ranked candidate lists for each proposal, notifies students of key milestones (ranking window, seriation results, confirmations), and allows companies to review and refine candidate selections, including excluding unsuitable candidates.

FR-4x: Company and User Management: This category encompasses user and organizational management across all roles. Coordinators validate new company registrations and associated legal protocols before proposals can be submitted. Company representatives manage their organization structure and employee accounts, keeping contacts and mentors for proposals up to date. The system also supports bulk user pre-registering (*e.g.*, student imports via CSV, manual registrations of professors and secretariat staff) and a controlled "*superuser*" capability, which allows coordinators to access user accounts for support purposes with full audit logging. Sub-coordinator roles can be assigned and revoked for specific editions to delegate proposal validation and feedback tasks.

FR-5x: Communication and Notifications: This category covers the communication mechanisms that replace the legacy system's unreliable email workflows. The platform automates the "*Call for Proposals*", sending active companies edition-specific information (deadline, edition type) and allows coordinators to publish news and announcements targeted at specific user roles. Critical events - such as new proposals, feedback requests, ranking deadlines and seriation results - generate both email and in-app notifications, using configurable templates and a dedicated transactional email service to improve deliverability. Coordinators can also send targeted email campaigns to selected user groups, with customizable subject and content.

FR-6x: Reporting and Data Export: This category provides the reporting and audit capabilities required for institutional oversight. Coordinators and administrator can export student lists, proposals, candidacies and final assignments in open formats (CSV, JSON), overcoming the legacy platform's interoperability limitations. The system maintains detailed audit log of critical actions (logins, proposal status, changes, permission updates, superuser access, bulk imports) with timestamps and user attribution, as well as a trace of proposal feedback exchanges between coordinators and companies. These records support accountability, compliance checks and dispute resolution.

FR-7x: Master's Dissertation Lifecycle Support: This category addresses the specific needs of Master's dissertations and engineering projects, turning previously unused "dead data" into fully supported workflows. The system distinguishes between scientific dissertation and industry-oriented engineering projects, allowing professors and companies to propose themes, manage assigned students and track progress.

Master's editions follow a lifecycle similar to undergraduate internships but include additional milestones such as topic approval and final submission/defense dates, reflecting the supervision and evaluation requirements of Master's programmes

3.2.4 Non-Functional Requirements

To address the technical debt of the legacy system and ensure the platform meets institutional and user expectations, the new system must adhere to the following quality attributes, organized according to ISO/IEC 25010 standards. Detailed specifications for all-non functional requirements are presented in Annex I, Tables I.8-I.15. The following subsections provide a summary of each category:

Security: Security requirements ensure that sensitive institutional and personal data is protected against unauthorized access. The platform enforces RLS at the database layer, so users only see data appropriate to their role and organization context, and all communication is encrypted over HTTPS/TLS to avoid the legacy system's unencrypted HTTP risk [20]. Passwords and credentials are stored using industry-standard hashing, and role-based access control (RBAC) with granular permissions limitations each user to actions consistent with their assigned role [48].

Auditability & Compliance: Auditability requirements guarantee that all critical actions can be traced for accountability, compliance and dispute resolution. The system records a tamper-evident audit log of key events - such as logins, data changes, status updates, permission assignments and superuser sessions - with precise timestamps and user attributions [15]. Proposal feedback exchanges between coordinators are also logged, and superuser activity is recorded in detail (who accessed whom, when and for how long), providing transparency over administrative interventions.

Reliability & Availability: This category ensures the platform remains operational during critical periods like proposal submission or seriation. A transactional email service is used to deliver time-sensitive notifications reliably, addressing the legacy system's chronic email failures. Automated backups of database, documents and configuration, combined with off-site replication, support recovery from data loss. Health monitoring and alerting for components such as the database, backup jobs and email service enable fast detection and response to incidents, reducing downtime.

Performance & Scalability: Performance requirements guarantee a responsive experience for users, even under peak load. User-facing pages such as dashboards, proposal lists and candidacy views are expected to load within a few seconds under normal conditions, while key dashboards support real-time updates via WebSockets or server-sent events, so changes appear without manual refresh. API endpoints must respond within defined latency targets for most requests, and rate limiting is

applied to protect the system from abuse and ensure fair use of resources across standard and bulk operations.

Usability & Accessibility: This category ensures the platform is easy to use and accessible, addressing the legacy system's poor and outdated user experience. The interface must be responsive on both desktop and mobile, with clear visual feedback on interactive elements (focus, hover, loading) so users understand the current state. Core workflows - such as registration, proposal submission and candidacy preference ranking - should be completable without prior training, supported by concise inline guidance and meaningful error messages.

Interoperability & Data Exchange: Interoperability requirements guarantee that data can flow reliably between the platform and other institutional tools. The system supports bulk user imports (*e.g.*, students via CSV, professors and secretariat staff) with validation, error reporting and rollback. Coordinators and administrators can export key datasets - students, proposals, seriation results, assignments - in standard formats like CSV or JSON, and a documented REST API allows third-party systems to query selected read-only data without compromising security.

Maintainability & Extensibility: This category ensures the platform can be evolved and maintained without excessive technical debt. The architecture is modular and layered (presentation, business logic, data access), allowing frontend, backend and database concerns to be updated independently. The codebase is supported by appropriate documentation (README, API docs, comments for complex logic) and by disciplined Git workflows with semantic versioning and meaningful commit history. Configuration is externalized via environment variables or configuration files, rather than hardcoded, enabling consistent deployment across development, staging and production environments.

Email Deliverability (Critical Legacy Fix): Given that unreliable email was one of the most severe issues in the legacy system, email deliverability is treated as a separate non-functional requirement. All outbound messages must use proper sender authentication to reduce spam classification and improve inbox placement. The platform integrates with a professional transactional email service that offers monitoring and bounce handling, providing real-time insight into delivery status. Email templates are tested before deployment to ensure correct rendering across common clients (Gmail, Outlook, Apple Mail) and on mobile devices.

3.3 Use Cases

Use case modeling is a requirements engineering technique that describes interactions between actors (stakeholders) and the system to achieve specific goals. This section

documents key use cases for the dissertation/internship management platform, representing critical workflows identified in the requirements' analysis. Each use case follows a standardized format including actors, preconditions, postconditions, main flow, and alternative flows.

3.3.1 Actors and Their Goals

Bases on the stakeholder analysis in Section 3.2.2, the platform involves the following actors:

Primary Actors (initiate use cases to achieve their own goals):

- *Student* - Seeks to discover proposals, rank preferences, and track candidacy status
- *Company Representative* - Aims to submit proposals and manage company participation
- *Professor* - Intends to propose dissertation themes and supervise students
- *Edition Coordinator* - Responsible for configuring editions and overseeing workflows

Secondary Actors (support the system but do not initiate the primary goal):

- *Secretariat* - Manage financial and legal documentation
- *System Administrator* - Maintains infrastructure and system health

3.3.2 High-Level Use Case Diagram

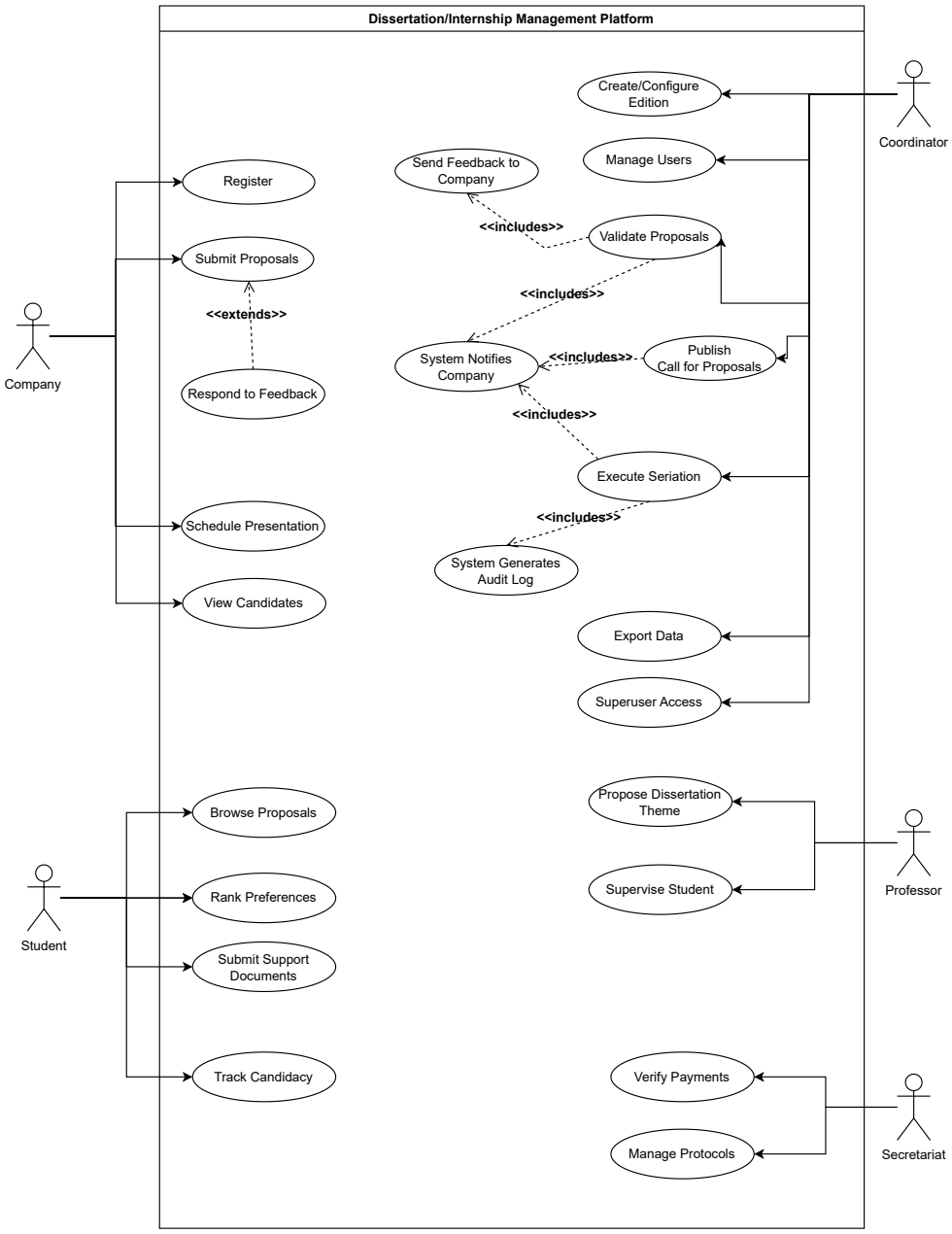

Figure 3.1: High-Level Use Case Diagram of the system with core functionalities

3.3.3 Detailed Use Case Specifications

This subsection documents the detailed specification of key use cases using the standard Cockburn template format: Use Case Name, Primary Actor(s), Secondary Actor(s), Preconditions, Postconditions, Main Flow, Alternative Flows [50]. The complete specifications for all use cases identified in this section are presented in Annex 2, Tables II.1-II.12.

The following describes the scope and organization of documented use cases:
Core Workflow Use Cases:

- **UC-01: Student Ranks Internship/Dissertation Preferences** - Central to the student candidacy phase; demonstrates the core functionality of the platform for students
- **UC-02: Coordinator Validates and Provides Feedback on Proposal** - Critical workflow; illustrates the iterative refinement loop between coordinator and company
- **UC-03: System Executes Student Seriation Algorithm** - Core business logic; represents the automated ranking that differentiates this platform form manual processes
- **UC-04: Company Responds to Coordinator Feedback and Resubmits Proposal** - Demonstrates bidirectional communication; shows how feedback loops improve proposal quality
- **UC-05: Edition Coordinator Creates New Edition** - Administrative setup; establishes the foundation for almost all the other workflows
- **UC-06: Company Representatives Registers and Submits Initial Proposal** - Entry point for external stakeholders; illustrates onboarding and validation workflows.
- **UC-07: Student Submits Support Documentation** - Optional workflow, triggered when a coordinator requires CV or recommendation letters

Secondary Workflow Use Cases:

- **UC-08: Company Views Assigned Candidates List and Schedule Interviews** - Post seriation activities; enables company participation in selection
- **UC-09: Professor Proposes Dissertation Theme** - Master's degree specific; operationalizes previously non-functional dissertation workflow
- **UC-10: Secretariat Verifies Payment and Finalizes Protocol** - Post-assignment; ensures legal and financial closure
- **UC-11: Edition Coordinator Publishes Call for Proposals** - Communication workflow; ensures all companies receive the invite to send proposals to the edition
- **UC-12: Administrator Performs Superuser Access for Student Support** - Administrative support; includes detailed audit logging requirements

Each use case specification includes:

- Actors - Primary (initiates the goal) and secondary (supports the goal) participants
- Preconditions - System state and permissions required before the use case can begin

- Postconditions - Guaranteed system state after the use case completes (success of failure)
- Main Flow - The primary, "happy path" sequence of steps
- Alternative Flows - Exception handling, edge cases, and variant paths (numbered A1, A2, A3, etc.) describing specific conditions where the main flow diverges.

Traceability to Requirements: Steps in the use case flows explicitly reference, when they have one, the corresponding Functional Requirement or Non-Functional Requirement (NFR-XX), ensuring complete traceability between user interactions and system capabilities.

3.3.4 Use Case Relationships and Extensions

The use case model also captures a small set of relationships that express shared or conditional behavior between use cases.

First, there is one extend relationship:

- Respond to Feedback extends Submit Proposals when the coordinator has requested clarifications. In this situation, the company follows an additional flow where it edits and resubmits the proposal instead of creating a completely new one.

Then, there are several include relationships, where one use case always performs another pas part of its normal execution:

- Validate Proposals includes Send Feedback to Company, because every validation ends with the coordinator communicating an acceptance, rejection, or clarification request.
- Execute Seriation includes System Generate Audit Log, since each seriation run must be recorded with its parameters and results for auditability.
- Publish Call for Proposals, Execute Seriation, and Validate Proposals each include System Notifies Company, reflecting that companies are automatically informed when a new edition is open for proposals, when seriation results are available, or when their proposals change status.

3.3.5 Summary of Use Cases

The use case model encompasses 12 primary use cases organized by the four main actors: Students, Companies, Coordinators, and supporting roles (Professors, Secretariat)

Student-Centric activities Students interact with the platform to browse and rank proposals, submit supporting documents if required, and track their candidacy status:

- UC-01: Rank Internship Preferences [FR-31]

- UC-07: Submit Supporting Documentation [FR-32]
- Track candidacy status after seriation [FR-34]

Company-Centric Activities Companies register, submit proposals, respond to coordinator feedback, and view ranked candidates:

- UC-06: Register and Submit Initial Proposal [FR-21, FR-41]
- UC-04: Respond to Coordinator Feedback and Resubmit Proposal [FR-22, FR-24]
- UC-08: View Ranked Candidates and Schedule Interviews [FR-35, FR-36]

Coordinator-Centric Activities Coordinators manage editions, validate proposals, publish calls for proposals, and execute seriation:

- UC-05: Create New Edition [FR-11, FR-12]
- UC-02: Validate Proposals [FR-22, FR-23]
- UC-03: Execute Seriation Algorithm [FR-33]

Professor-Centric Activities Professors primarily propose dissertation themes and supervise students.

- UC-09: Propose Dissertation Theme [FR-71, FR-72]

Cross-Functional Activities Supporting workflows for academic supervision, financial closure, and administrative support:

- UC-10: Verify Payments and Finalize Protocol [FR-41]
- UC-12: Superuser Access for Student Support [FR-44, NFR-AUD-4]

Conclusion

Complete detailed specifications for all 12 use cases - including preconditions, postconditions, main flows, and alternative flows - are documented in Annex 2, Tables II.1-II.12. These specifications will serve as input for the system architecture design and form the basis for implementation and testing.

4

Solution Proposal

4.1 Proposed System Architecture

The proposed platform follows a three-layer architecture that separates presentation, application logic and data persistence, while integrating technologies justified in Chapter 2.

At the presentation layer, a React/Next.js single-page application provides role-specific dashboards for students, company representatives, professors, edition coordinators, secretariat staff and system administrators. Next.js handles routing and page rendering (SSR/SSG), while React components implement reusable views such as proposal lists, candidacy tables and configuration forms.

The application layer is implemented as a set of backend services that expose a REST API consumed by the frontend. In the prototyping phase, these services are backed by Supabase (authentication, simple CRUD operations, real-time subscriptions), which allows rapid validation of core workflows. In the target architecture, a Spring Boot backend takes over the main business logic: proposal validation and feedback, student seriation, user and company management, protocol verification, notification orchestration and audit logging. Cross-cutting concerns such as authentication/authorization, email delivery and WebSocket-based real-time updates are handled centrally in this layer.

The data layer is built on PostgreSQL, using the relational schema that is described in Section 4.3. Core entities (Edition, User, Student, Company, Proposal, Candidacy, etc.) are linked through foreign keys to guarantee consistency, while RLS enforces fine-grained access control. JSON/JSONB columns are used for semi-structured data such as email templates and feedback metadata.

External services complete the architecture: a transactional email provider is responsible for reliable message delivery; object storage is used for uploaded documents (CVs, protocols, reports). This structure allows the initial AI-assisted prototypes to evolve into a maintainable, secure platform without discarding early work.

TODO: diagram maybe vai para os anexos

4.2 Process Modelling (BPMN)

TODO: mapping of workflows with bpmn

4.3 Data Model (ERD)

TODO: presentations of the ER diagram (db diagram) e definition of the entities there

4.4 Work Plan

TODO: maybe a cronogram (gantt?) with the development phases, tests, docs writing, risks and mitigations

Bibliography

[1] AlSalah. *Benefits and Drawbacks of Using Backend as a Service*. Point of SaaS. 2025. URL: https://pointofsaas.com/benefits-and-drawbacks-of-using-backend-as-a-service-3/ (cit. on p. 8).

[2] Anna Arteeva. *Choosing your AI prototyping stack: Lovable, v0, Bolt, Replit, Cursor, Magic Patterns compared*. 2025. URL: https://annaarteeva.medium.com/choosing-your-ai-prototyping-stack-lovable-v0-bolt-replit-cursor-magic-patterns-compared-9a5194f163e9 (cit. on pp. 16, 17).

[3] *App Router (Next.js)*. Next.js Documentation. Vercel. 2026. URL: https://nextjs.org/docs/app (cit. on p. 13).

[4] Ayush Sharma. *Difference between SQL and NoSQL*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/sql/difference-between-sql-and-nosql/ (cit. on p. 9).

[5] B. Biskupski and A. Pytko-Włodarczyk. *Backend as a Service (BaaS) vs. custom backend: Which one to choose and why?* Accessed: 2026-01-17. 2019. URL: https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend (cit. on p. 8).

[6] *Chapter 15. Envers*. Hibernate Community. 2013. URL: https://docs.hibernate.org/orm/4.3/devguide/en-US/html/ch15.html (cit. on p. 9).

[7] P. P. Chen. "The Entity-Relationship Model: Toward a Unified View of Data". In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36 (cit. on p. 4).

[8] Cloudflare, Inc. *What is vibe coding?* 2025. URL: https://www.perplexity.ai/search/add-this-as-a-citation-in-late-dWWatfeETdCTpmIFNr6Jbg?sm=d (cit. on p. 15).

[9] W. contributors. *Vibe coding - chat based AI-assisted software development definition*. Accessed: 2026-01-17. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding (cit. on p. 4).

[10] David Clayton. *How to Improve Deliverability of Transactional Emails*. SMTP. 2023. URL: https://www.smtp.com/blog/transactional-email/improve-deliverability-of-transactional-emails/ (cit. on pp. 14, 21).

[11] Elena. *Why Using Legacy PHP Versions Makes Your Website Vulnerable*. Fastcomet. 2022. URL: https://www.fastcomet.com/blog/legacy-php-versions-makes-your-website-vulnerable (cit. on p. 21).

[12] *Embedded JavaScript templating Documentation*. EJS. 2026. URL: https://ejs.co/#docs (cit. on p. 9).

[13] *Express.js Documentation*. Express.js Foundation. 2026. URL: https://expressjs.com/ (cit. on p. 9).

[14] Figma. *What is Vibe Coding? Everything You Need to Know*. 2025. URL: https://www.figma.com/resource-library/what-is-vibe-coding/ (cit. on p. 7).

[15] Fortra. *Audit Log Best Practices for Security & Compliance*. 2024. URL: https://www.fortra.com/blog/audit-log-best-practices-security-compliance (cit. on p. 27).

[16] O. Glib. *Backend as a Service use Cases: How to apply*. Accessed: 2026-01-17. 2024. URL: https://acropolium.com/blog/baas-use-cases/ (cit. on p. 7).

[17] *Handlebars.js Documentation*. Handlebars.js. 2026. URL: https://handlebarsjs.com/ (cit. on p. 9).

[18] IBM Corporation. *Foreign key (referential) constraints*. IBM DB2 12.1.x Documentation. Accessed: 2026-01-21. 2026. URL: https://www.ibm.com/docs/en/db2/12.1.x?topic=constraints-foreign-key-referential (cit. on p. 10).

[19] Jasmin Tankić. *Unlocking the Power of Postgres Listen/Notify: Building a Scalable Solution With Spring Boot Integration*. DZone. 2023. URL: https://dzone.com/articles/leveraging-postgres-listennotify-in-spring-boot (cit. on p. 14).

[20] kartik. *Difference between http:// and https://*. GeeksforGeeks. 2026. URL: https://www.geeksforgeeks.org/computer-networks/difference-between-http-and-https/ (cit. on pp. 21, 27).

[21] Ketevan Bostoganashvili and Piotr Malek. *I Compared 7 Best Transactional Email Services: Here's What I Found*. mailtrap. 2025. URL: https://mailtrap.io/blog/transactional-email-services/ (cit. on p. 14).

[22] Mercy Tanga. *SSR vs. SSG in Next.js: Differences, Advantages, and Use Cases*. strapi. 2024. URL: https://strapi.io/blog/ssr-vs-ssg-in-nextjs-differences-advantages-and-use-cases (cit. on p. 13).

[23] Microsoft. *Joins (SQL Server)*. SQL Server Documentation. 2025. URL: https://learn.microsoft.com/en-us/sql/relational-databases/performance/joins?view=sql-server-ver17 (cit. on p. 10).

[24] Nafisa Anjum. *ReactJS Reconciliation*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/reactjs-reconciliation/ (cit. on p. 12).

[25] *Next.js Docs*. Vercel. 2026. URL: https://nextjs.org/docs (cit. on p. 13).

[26] *Node.js Documentation*. Node.js Foundation. 2026. URL: https://nodejs.org/en/docs/ (cit. on p. 9).

[27] Object Management Group. *Business Process Model And Notation (BPMN) Version 2.0*. OMG Document Number: formal/2011-01-03. Version 2.0. Object Management Group, 2011 (cit. on p. 4).

[28] Oskar Dudycz. *PostgreSQL JSONB - Powerful Storage for Semi-Structured Data*. 2025. URL: https://www.architecture-weekly.com/p/postgresql-jsonb-powerful-storage (cit. on p. 10).

[29] Paul Bratslavsky. *10 Best WYSIWYG Editors for 2025*. strapi. 2025. URL: https://strapi.io/blog/best-wysiwyg-editors (cit. on p. 14).

[30] Pedro Alonso. *PostgreSQL LISTEN/NOTIFY: Real-Time Without the Message Broker*. 2025. URL: https://www.pedroalonso.net/blog/postgres-listen-notify-real-time/ (cit. on p. 13).

[31] Philippe Sevestre. *Using PostgreSQL as a Message Broker*. Baeldung. 2023. URL: https://www.baeldung.com/spring-postgresql-message-broker (cit. on p. 13).

[32] PostgreSQL Global Development Group. *8.14. JSON Types*. PostgreSQL 18 Documentation. 2026. URL: https://www.postgresql.org/docs/current/datatype-json.html (cit. on p. 10).

[33] Pranjal Nanda. *React Component Based Architecture*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/react-component-based-architecture/ (cit. on p. 12).

[34] K. Pykes. *What Are ACID Transactions? A Complete Guide for Beginners*. Datacamp. 2025. URL: https://www.datacamp.com/blog/acid-transactions (cit. on p. 9).

[35] React Team. *React*. Official React Documentation. Meta. 2025. URL: https://react.dev/ (cit. on pp. 11, 12).

[36] React Team. *React*. Meta Open Source Projects. Meta Open Source. 2026. URL: https://opensource.fb.com/projects/react/ (visited on 2026-01-23) (cit. on p. 12).

[37] Sachin Chaurasiya. *Tailwind CSS: Utility-First Styling for Rapid UI Development*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/blogs/tailwind-css-utility-first-styling-for-rapid-ui-development/ (cit. on p. 19).

[38] J. Schibelli. *Vibe Coding: How AI is Transforming Software Development*. Accessed: 2026-01-17. 2025. URL: https://dev.to/johnschibelli/the-emergence-of-vibe-coding-how-ai-is-revolutionizing-software-development-4275 (cit. on pp. 7, 15).

[39] K. Schwaber and J. Sutherland. *The 2020 Scrum Guide*. Accessed: 2026-01-17. 2020. URL: https://scrumguides.org/ (cit. on p. 3).

[40] R. H. Shaikh. *Referential Integrity: Why It's Vital for Databases*. acceldata. 2024. URL: https://www.acceldata.io/blog/referential-integrity-why-its-vital-for-databases (cit. on p. 10).

[41] Spring Team. *Spring Boot Reference Documentation*. 3.x. Version 3.x. VMware, Inc. 2026. URL: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ (cit. on pp. 4, 9, 14).

[42] Supabase. *Auth Overview*. 2026. URL: https://supabase.com/docs/guides/auth (cit. on p. 8).

[43] Supabase. *Edge Functions*. 2026. URL: https://supabase.com/docs/guides/functions (cit. on p. 8).

[44] Supabase. *Realtime*. 2026. URL: https://supabase.com/docs/guides/realtime (cit. on p. 13).

[45] Supabase. *REST API Overview*. 2026. URL: https://supabase.com/docs/guides/api#rest-api-overview (cit. on p. 8).

[46] Supabase. *Row Level Security*. 2026. URL: https://supabase.com/docs/guides/database/postgres/row-level-security (cit. on p. 8).

[47] Supabase. *Supabase: The Open Source Firebase Alternative*. https://github.com/supabase/supabase. Project README. 2019 (cit. on pp. 4, 8).

[48] Susan Stapleton. *Role-Based Access Control (RBAC) - A Comprehensive Guide*. pathlock. 2025. URL: https://pathlock.com/blog/role-based-access-control-rbac/ (cit. on p. 27).

[49] Tomasz Dąbrowski. *Using Spring Boot for WebSocket Implementation with STOMP*. Toptal. 2026. URL: https://www.toptal.com/developers/java/stomp-spring-boot-websocket (cit. on p. 14).

[50] Visual Paradigm. *A Comprehensive Guide to Use Case Modeling*. 2023. URL: https://guides.visual-paradigm.com/a-comprehensive-guide-to-use-case-modeling/ (cit. on p. 30).

[51] *What's the Difference Between an ACID and a BASE Database?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-acid-and-base-database/ (cit. on p. 9).

[52] *What's the Difference Between Relational and Non-relational Databases?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-relational-and-non-relational-databases/ (cit. on pp. 9, 11).

I

Annex 1 - Functional and Non-Functional Requirements

Functional Requirements

Table I.1: Functional Requirements - Edition and Configuration Management

ID	Description
FR-11	The system shall allow Edition Coordinators to create new academic editions, specifying edition type (Undergraduate Internship vs Master's Dissertation) and defining key timelines (proposal submissions deadline, interview periods, finalization deadline)
FR-12	The system shall allow Edition Coordinators to define and manage time slots for company proposal presentations, which companies can subsequently view and book during available windows
FR-13	The system shall enable Edition Coordinators to create and edit reusable email templates within the platform's interface, supporting dynamic variables (student names, company names, deadlines) and content-specific formatting for different communication stages.
FR-14	The system shall allow Edition Coordinators to create and maintain edition-specific informational pages (FAQs, Regulations, Contact Information, Important Dates) using a modern WYSIWYG editor.
FR-15	The system shall allow Edition Coordinators to publish news and announcements target at specific user roles (e.g., news visible only to Students, Companies, or Professors) within a given edition or platform-wide

Table I.2: Functional Requirements - Proposal Submission and Classification

ID	Description
FR-21	The system shall allow Companies and Professors to submit detailed proposals or dissertation themes, including attributes such as title, objectives, technical requirements, location, benefits, and supervisor information.
FR-22	The system shall implement a feedback loop allowing Edition Coordinators to request clarifications on specific aspects of proposals without full rejection, triggering email notifications to the author for editing and resubmission.
FR-23	The system shall allow Edition Coordinators to accept, conditionally accept, or reject proposals with detailed feedback comments.
FR-24	The system shall allow Companies to import or duplicate proposals from previous editions into the current edition, with the option to modify details before resubmission.
FR-25	The system shall allow Companies to indicate their intent to give oral presentations of their proposals and select available time slots directly within the platform.
FR-26	The system shall notify Companies of important deadlines (proposal submission closure, feedback response deadline, presentation scheduling deadline) via email and in-app notifications.
FR-27	The system shall notify Companies when a new edition opens in which they are eligible to submit proposals (Call for Proposals).

Table I.3: Functional Requirements - Student Candidacies and Seriation

ID	Description
FR-31	The system shall allow Students to view all available proposals for their enrolled edition and rank a specified number of preferred proposals in preference order.
FR-32	The system shall allow Students to submit supporting documentation (CV, motivation letter, recommendation letter) when registering interest in specific proposals, if required by the coordinator.
FR-33	The system shall execute an automated ranking algorithm based on predefine criteria (e.g. ECTS completed, grade average, coordinator-defined weights) to produce a seriation list of candidates for each proposal.
FR-34	The system shall notify Students of important timeline events via email and in-app notifications (registration opening, ranking deadline, seriation results, attribution confirmation)
FR-35	The system shall allow Companies to view the list of serialized (ranked) students for their proposals and manage candidate selections throughout the interview and confirmation phases.
FR-36	The system shall allow Companies to exclude or deselect candidates from their proposal's ranked list if needed.

Table I.4: Functional Requirements - Company and User Management

ID	Description
FR-41	The system shall allow Edition Coordinators to validate new company registrations, ensuring they sign the necessary legal protocols before submitting proposals
FR-42	The system shall allow Company Representatives to manage their organizational hierarchy, including parent company designation and sub-entities/departments, and to update company contact details and registration information
FR-43	The system shall allow Company Representatives to manage company employee accounts (add, remove, update) and designate which employees are responsible for specific proposals or serve as mentors.
FR-44	The system shall provide "Superuser" functionality, allowing Edition Coordinators to temporarily access other user accounts using only their user's identifier (student number or username) for support and troubleshooting purposes, with full audit logging of such access.
FR-45	The system shall allow Edition Coordinators to pre-register users (Students via CSV bulk import or manual entry, Professors, Secretariat staff) with role-based permissions.
FR-46	The system shall allow Edition Coordinators to assign or revoke the Sub-Coordinators role to Professors for specific reasons.

Table I.5: Functional Requirements - Communication and Notifications

ID	Description
FR-51	The system shall automate the "Call for Proposals" process, sending notifications to all registered companies (that have an active status) at the start of each edition, specifying submission deadlines and edition type.
FR-52	The system shall allow the publication of news and announcements targeted at specific user roles (e.g., news visible only to Students)
FR-53	The system shall enforce email deliverability best practices to minimize the likelihood of notifications being flagged as spam.
FR-54	The system shall support both email and in-app notifications for critical events (new proposals, feedback requests, deadline reminders, seriation results)
FR-55	The system shall allow Edition Coordinators to send targeted email communications to specific user groups (e.g., all enrolled Students, all registered Companies) with customizable subject and body content.

Table I.6: Functional Requirements - Reporting and Data Export

ID	Description
FR-61	The system shall allow Edition Coordinators and Administrators to export lists of students, proposals, candidacies, and final assignments in open standard formats (CSV, JSON).
FR-62	The system shall maintain detailed audit logs of critical system actions (user logins/logouts, proposal status changes, superuser access, permission modifications, bulk imports), with timestamps and user attribution.
FR-63	The system shall provide auditable records of all proposal feedback exchanges (coordinator feedback, company responses, timestamp of modifications) for compliance and dispute resolution.

Table I.7: Functional Requirements - Master's Dissertation Lifecycle Support

ID	Description
FR-71	The system shall support specific workflows for Master's degrees, including the distinction between scientific dissertations and engineering projects.
FR-72	The system shall allow Professors to propose scientific dissertation themes for Master's programs and manage their list of proposed themes and assigned students.
FR-73	The system shall enable tracking of dissertation progress and completing status, including milestones such as topic approval, and final submission/defense.
FR-74	The system should have edition for the Master's degree scientific dissertation and engineering projects that work the same way as the undergraduate internships, with just some changes.

Non-Functional Requirements

Table I.8: Non-Functional Requirements - Security

ID	Description
NFR-SEC-1	The system shall implement Row-Level Security at the database level to enforce strict data isolation, ensuring users can only access data they are authorized to view (e.g., companies see only their own proposals and candidates)
NFR-SEC-2	All data in transit shall be encrypted using HTTPS/TLS, preventing interception of sensitive information during transmission.
NFR-SEC-3	Password and sensitive credentials shall be hashed using industry-standard algorithms (bcrypt, PBKDF2, or Argon2) and shall never be stored in plaintext.
NFR-SEC-4	The system shall enforce role-based access control (RBAC) with granular permission assignment, ensuring users can only perform actions consistent with their assigned role.

Table I.9: Non-Functional Requirements - Auditability & Compliance

ID	Description
NFR-AUD-1	The system shall record an immutable, tamper-evident audit log of all critical actions (user logins/logouts, data modifications, status changes, permission assignments, superuser access) with precise timestamps and user attribution.
NFR-AUD-2	Audit logs shall be retained for long time to satisfy institutional compliance and legal hold requirements.
NFR-AUD-3	The system shall provide auditable records of all proposals feedback exchanges (coordinator feedback, company responses, timestamp of submission/modification) enabling dispute resolution and quality review.
NFR-AUD-4	Superuser access shall be logged in detail, including the identity of the accessing administrator, the target user account, timestamp, and duration of the session.

Table I.10: Non-Functional Requirements - Reliability & Availability

ID	Description
NFR-REL-1	The transaction email service shall guarantee a good deliverability rate to ensure time-sensitive notifications (e.g., proposals deadlines, seriation results) reach intended recipients reliably
NFR-REL-2	The system shall implement automated backups of all critical data (database, uploaded documents, configuration) with off-site replication to enable recovery in case of data loss.
NFR-REL-3	The system shall implement health monitoring and automated alerts for critical failures (database unavailability, backup failures, email service degradation), enabling rapid response by administrative staff.

Table I.11: Non-Functional Requirements - Performance & Scalability

ID	Description
NFR-PERF-1	All user-facing pages (dashboards, proposal lists, candidacy tracking) shall load within 3 seconds under normal load conditions.
NFR-PERF-2	Critical dashboards (Coordinator Edition Dashboard, Company Proposal Tracker, Student Status View) shall support real-time updates using WebSocket subscriptions or server-sent events, reflecting database changes without manual page refresh.
NFR-PERF-3	API endpoints shall respond within 500 ms for 95% of requests under typical load, and within 2 seconds for 99% of requests.
NFR-PERF-4	The system shall implement API rate limiting to protect against denial-of-service attacks and ensure fair resource allocation, limiting each user/API client to reasonable thresholds (e.g. 100 requests/minute for standard users, 300 requests/minute for bulk operations).

Table I.12: Non-Functional Requirements - Usability & Accessibility

ID	Description
NFR-USAB-1	The user interface shall be responsive and mobile-friendly, providing an optimal viewing experience on desktop (1920x1080) and mobile devices (320x568).
NFR-USAB-2	All interactive elements shall provide clear visual feedback (hover states, focus indicators, loading states) enabling users to understand the current application state.
NFR-USAB-3	New user workflows (registration, proposal submission, candidacy ranking) shall be completable without raining or external documentation, with clear inline guidance and error messages.

Table I.13: Non-Functional Requirements - Interoperability & Data Exchange

ID	Description
NFR-INTER-1	The system shall facilitate bulk import of users (Students, Professors, Secretariat) via CSV files with validation, error reporting, and rollback capabilities.
NFR-INTER-2	The system shall support export reporting data (student lists, proposals, seriation results, internship assignments) in standard formats (CSV, JSON) for use in external tools
NFR-INTER-3	The system shall provide a documented REST API for institutional integration, enabling third-party systems to query limited, read-only data.

Table I.14: Non-Functional Requirements - Maintainability & Extensibility

ID	Description
NFR-MAINT-1	The system architecture shall be modular and layered (presentation, business logic, data access), enabling independent updates to the frontend, backend, or database without cascading failures.
NFR-MAINT-2	The codebase shall be documented with README files, API documentation (Swagger/OpenAPI), and inline comments for complex business logic, enabling new developers to understand and modify the system.
NFR-MAINT-3	The system shall support semantic versioning and use Git-based version control with meaningful commit messages and branch-based workflows for traceability.
NFR-MAINT-4	Configuration (database URLs, email credentials, feature flags) shall be externalized using environment variables or configuration files, not hardcoded, to enable deployment across development, staging and production environments.

Table I.15: Non-Functional Requirements - Email Deliverability (Critical Legacy Fix)

ID	Description
NFR-EMAIL-1	All outbound emails shall implement sender authentication protocols to minimize spam flagging
NFR-EMAIL-2	The system shall integrate with a professional transaction email service with built-in deliverability monitoring and bounce handling.
NFR-EMAIL-3	Email templates shall be tested and validate before deployment to ensure correct rendering across major email clients (Gmail, Outlook, Apple Mail) and responsive display on mobile.

II

Annex 2 - Use Cases Specifications

Use Cases

Table II.1: Use Case Specification: Proposal Ranking

Use Case ID	UC-01
Use Case Name	Student Ranks Internship/Dissertation Preferences
Primary Actor	Student
Secondary Actors	Edition Coordinator, System
Preconditions	- Student is enrolled in current edition [FR-31]. - Proposal preference ranking period is open (not before start date, not after deadline). - At least one proposal is available for the edition.
Postconditions	- Student's preference ranking is saved in the system. - Confirmation message is displayed to the student. - Coordinator can retrieve data for seriation.
Main Flow	1. Student logs in and navigates to the edition they are enrolled in. 2. System displays all available proposals for the edition. 3. Student views details of each proposal (objectives, requirements, company, benefits, etc.). 4. Student selects and ranks a specified number of proposals in priority order. 5. System validates the number of proposals does not exceed the limit. 6. Student submits the ranking. 7. System confirms successful submission and displays a confirmation message. 8. System stores the ranking with timestamp [NFR-AUD-1].
Alternative Flows	**A1. Post-Deadline Attempt:** 1. System prevents submission; displays "Choosing period has closed". 2. Use case terminates. **A2. System Failure:** 1. System displays error, prompts user to try again. 2. Student's previous preference ranking remains unchanged (rollback).

Table II.2: Use Case Specification: Coordinator Validates and Provides Feedback on Proposal

Use Case ID	UC-02
Use Case Name	Coordinator Validates and Provides Feedback on Proposal
Primary Actor	Edition Coordinator
Secondary Actors	Company Representative, System, Email Service
Preconditions	- A new proposal has been submitted by a Company or Professor. - Edition Coordinator is assigned to this edition and has validation permission [FR-22]. - The proposal review period is active.
Postconditions	- Proposal feedback is recorded in the system. - Company Representative is notified via email and in-app notification. - Proposal status is updated (pending clarification, accepted, or rejected). - Audit log records the coordinator's action with timestamp.
Main Flow	1. Edition Coordinator logs in and navigates to "Validate Proposals" page. 2. System displays list of submitted proposals awaiting validation. 3. Coordinator selects proposal to review. 4. System displays full proposal details with all submitted information. 5. Coordinator review proposal and identifies areas requiring clarification (or accepts/rejects). 6. If clarification needed: Coordinator writes specific feedback in the feedback form. 7. Coordinator selects "Request clarification" and submits [FR-22]. 8. System updates proposal status to "Awaiting Clarification". 9. System sends email to Company Representative with feedback details [FR-53]. 10. System posts in-app notification to Company dashboard [FR-54]. 11. Audit log records: Coordinator name, action, feedback content, timestamp [FR-62].

Alternative Flows	**A1. Coordinator accepts proposal without changes:** 1. Step 6 is skipped. 2. Coordinator selects "Accept Proposal". 3. Proposal becomes visible to students for choosing. 4. Company is notified of approval via email and in-app notification. **A2. Coordinator rejects proposal:** 1. Coordinator selects "Reject Proposal" and provides a rejection reason 2. System updates status to "Rejected". 3. Company is notified with rejection reason via email [FR-53]. 4. Proposal is not visible to students **A3. Company fails to respond to clarification request within deadline:** 1. System sends automated reminder email 2 days before deadline. 2. After deadline, proposal automatically moves to "Deadline Exceeded" status 3. Coordinator can re-request clarification or reject.

Table II.3: Use Case Specification: System Executes Student Seriation Algorithm

Use Case ID	UC-03
Use Case Name	System Executes Student Seriation Algorithm
Primary Actor	System
Secondary Actors	Edition Coordinator, Student, Company
Preconditions	- The seriation period has begun (coordinator has triggered seriation) - All students have submitted their preference rankings [FR-31] - All accepted proposals are locked (no further modifications allowed) - Pre-defined seriation criteria have been configured by coordinator.
Postconditions	- Seriation algorithm completes successfully - Assignments lists are generated for each proposal [FR-35] - Students are notified of seriation results via email and in-app notification - Companies receive the list of students assigned for each of their proposals - Audit log records seriation execution details.

Main Flow	1. Edition Coordinator logs in and navigates to "Run Seriation" 2. System displays a summary: number of students, number of proposals, seriation parameters (ECTS weight: X%, Average Grade weight: Y%) 3. Coordinator reviews and confirms parameters 4. Coordinator clicks "Execute Seriation" 5. System executes the seriation algorithm [FR-33]: • For each proposal, retrieves all student preference rankings where this proposal is ranked • Calculates a composite score for each student: (ECTSweight x studentECTS + AVGGradeweight x studentAVGGrade) • Sorts students by composite score in descending order • Produces a ranked list of candidates for each proposal 6. System generates proposal-to-candidate assignment results 7. System sends email notifications to all students listing their seriation results [FR-34] 8. System sends email notifications to all companies with their ranked candidate lists [FR-35] 9. System displays "Seriation Complete" confirmation to coordinator 10. Audit log records: seriation execution time, number of students processed, algorithm parameters used [FR-62]
Alternative Flows	**A1. Algorithm detects data inconsistency (e.g., student has invalid Average Grade)** 1. System logs error and notifies coordinator 2. Seriation halts; coordinator must resolve data issue and retry **A2. No students ranked a particular proposal** 1. System marks that proposal with "No Candidates" status 2. Proposal is excluded from seriation results

Table II.4: Use Case Specification: Company Responds to Coordinator Feedback and Resubmits Proposal

Use Case ID	UC-04
Use Case Name	Company Responds to Coordinator Feedback and Resubmits Proposal
Primary Actor	Company Representative
Secondary Actors	Edition Coordinator, System
Preconditions	- Coordinator has requested clarification on company's proposal [UC-02, A3]. - Company has not yet exceeded the response deadline. - Company Representative is logged in.
Postconditions	- Modified proposal is resubmitted. - Coordinator is notified of resubmission. - Proposal is returned to "Under Review" status. - Feedback exchange is recorded in audit trail [FR-63].
Main Flow	1. Company Representative receives email notification of requested clarification. 2. Representative logs in to the platform and navigates to "My Proposals". 3. System displays the proposal marked "Awaiting Clarification" with coordinator's feedback highlighted. 4. Representative reviews feedback and modifies proposal details (e.g., clarifies location, adds requirements). 5. Representative adds a response comment addressing each point of feedback. 6. Representative clicks "Resubmit Proposal". 7. System validates modified proposal for completeness. 8. System updates proposal status to "Under Review". 9. System sends email to Edition Coordinator with notification of resubmission [FR-54]. 10. Audit log records: resubmission timestamp, modification details, company response comment [FR-63].

Alternative Flows	**A1. Company does not respond before the deadline:** 1. System sends reminder email 2 days before deadline. 2. After deadline passes, proposal is automatically marked "Deadline Exceeded". 3. Coordinator can choose to reject or grant extension. **A2. Company's modifications do not resolve the coordinator's concerns:** 1. Coordinator reviews resubmitted proposal (follows UC-02 main flow). 2. If still inadequate, coordinator can request additional clarification (cycle repeats).

Table II.5: Use Case Specification: Edition Coordinator Creates New Edition

Use Case ID	UC-05
Use Case Name	Edition Coordinator Creates New Edition
Primary Actor	Edition Coordinator
Secondary Actors	System
Preconditions	- Coordinator is authenticated and authorized as edition coordinator. - At least one academic program and default configuration template exist.
Postconditions	- A new edition record is created with status “Draft” or “Configured”. - Key parameters (dates, proposal limits, seriation rules) are stored. - Edition is available for later activation and publication.
Main Flow	1. Coordinator accesses the “Manage Editions” area. 2. System displays a list of existing editions and an option to create a new one. 3. Coordinator selects “Create Edition”. 4. System presents a form to define edition metadata (name, academic year, degree type, description). 5. Coordinator fills in dates (proposal submission window, student ranking window, seriation execution window). 6. Coordinator defines proposal constraints (max proposals per company, per professor, per student). 7. Coordinator configures seriation parameters (e.g., average grade and ECTS weights, tie-breaking rules). 8. Coordinator saves the edition. 9. System validates the data, creates the edition in “Draft/Configured” status and confirms creation.

Alternative Flows	**A1. Invalid or missing data:** 1. At step 9, validation fails (e.g., end date before start date). 2. System shows specific validation errors and highlights problematic fields. 3. Coordinator corrects the data and retries saving. **A2. Duplicate edition:** 1. System detects an existing edition with conflicting key identifiers (e.g., same academic year and cycle). 2. System warns coordinator and suggests editing the existing edition instead. 3. Coordinator either cancels or changes identifiers and saves again.

Table II.6: Use Case Specification: Company Representative Registers and Submits Initial Proposal

Use Case ID	UC-06
Use Case Name	Company Representative Registers and Submits Initial Proposal
Primary Actor	Company Representative
Secondary Actors	System, Secretariat (for protocol validation), Coordinator
Preconditions	- Call for proposals for at least one edition is open. - Representative has a valid email address.
Postconditions	- A new company account (or user under an existing company) is created and marked as “Pending Validation” or “Active”. - At least one proposal is stored with status “Submitted” or “Pending Validation”. - Coordinator can see the proposal in the validation queue.
Main Flow	1. Representative accesses the platform and chooses “Register Company / Representative”. 2. System displays a registration form (company info, contact details, legal identifiers). 3. Representative fills in the form and submits. 4. System validates data, creates the company (or links user to existing company), and sends an activation email. 5. Representative activates the account via the email link and logs in. 6. Representative navigates to “Submit Proposals”. 7. System shows available editions and their submission windows. 8. Representative selects an edition and fills in proposal data (title, description, objectives, requirements, location, compensation/benefits). 9. Representative submits the proposal. 10. System validates the proposal, stores it with status “Submitted/Pending Validation”, and shows a confirmation.

Alternative Flows	**A1. Company already exists:** 1. At step 3, system detects that the company exists. 2. System offers to link the new user to the existing company instead of creating a new one. 3. Representative confirms and proceeds; account is created under that company. **A2. Registration incomplete:** 1. Representative closes the browser or cancels before activation. 2. System keeps a temporary record, which can be completed later via emailed link or is purged after a timeout. **A3. Proposal submitted after deadline:** 1. At step 8, if the edition's proposal window is closed, system blocks submission. 2. System shows an error and suggests choosing another open edition (if available).

Table II.7: Use Case Specification: Student Submits Support Documentation

Use Case ID	UC-07
Use Case Name	Student Submits Support Documentation
Primary Actor	Student
Secondary Actors	Coordinator, System
Preconditions	- Student is authenticated and enrolled in an edition. - Student has already created or is creating a ranking of proposals. - Coordinator or edition configuration requires supporting documents for that edition or specific proposals.
Postconditions	- Supporting documents (e.g., CV, motivation letter, recommendations) are uploaded and linked to the student's candidacies. - Coordinator and companies (where applicable) can access documents during seriation and evaluation.
Main Flow	1. Student opens the "My Candidacies / Rankings" page. 2. System indicates which candidacies or editions require supporting documents. 3. Student selects a candidacy requiring documents. 4. System displays required and optional document types (e.g., CV required, motivation letter optional). 5. Student uploads the requested files and confirms. 6. System validates file types and sizes, stores them securely, and links them to the corresponding candidacy. 7. System shows a confirmation and updates the candidacy status to "Documents Submitted".

Alternative Flows	**A1. Missing required document:** 1. At step 5, student omits a required file. 2. System highlights the missing document and prevents completion until it is provided. **A2. Invalid file:** 1. Uploaded file exceeds size limit or uses a forbidden type. 2. System rejects that file and informs the student of allowed types and limits. **A3. Update documents:** 1. Before the document deadline, student returns to the candidacy. 2. System allows replacing or adding documents. 3. New versions are stored and previous versions are either archived or overwritten.

Table II.8: Use Case Specification: Company Views Assigned Candidates List and Schedules Interviews

Use Case ID	UC-08
Use Case Name	Company Views Assigned Candidates List and Schedules Interviews
Primary Actor	Company Representative
Secondary Actors	Coordinator, System
Preconditions	- Seriation for the relevant edition has been executed successfully. - Company has at least one accepted proposal in that edition. - Representative is authenticated and linked to that company.
Postconditions	- Representative can see ranked or assigned candidates per proposal.
Main Flow	1. Representative logs in and navigates to "My Proposals / Candidates". 2. System lists the company's proposals for the edition and their current seriation status. 3. Representative selects a proposal. 4. System displays the ranked candidate list or assigned students, including basic profile data allowed by policy. 5. Representative optionally selects one or more candidates get their contacts to schedule interview outside the platform.
Alternative Flows	**A1. No candidates available:** 1. At step 4, system indicates that no candidates were ranked for that proposal. 2. Representative may close the view or contact the coordinator outside this use case.

Table II.9: Use Case Specification: Professor Proposes Dissertation Theme

Use Case ID	UC-09
Use Case Name	Professor Proposes Dissertation Theme
Primary Actor	Professor
Secondary Actors	Coordinator, System
Preconditions	- A Master's dissertation edition is open for receiving internal proposals. - Professor is authenticated and has permission to submit themes.
Postconditions	- A new dissertation theme is stored with status "Submitted" or "Pending Validation". - Coordinator can review and publish the theme to students.
Main Flow	1. Professor logs in and accesses "Dissertation Themes / New Theme". 2. System shows a form for theme details (title, abstract, objectives, prerequisites, research area, expected outcomes). 3. Professor fills in details, selects the relevant edition, and optionally co-supervisors. 4. Professor submits the theme. 5. System validates required fields and stores the theme. 6. Theme appears in the coordinator's validation queue.
Alternative Flows	**A1. Edition closed:** 1. At step 3, if the dissertation edition is not accepting themes, system blocks submission. 2. System informs the professor and shows allowed submission periods. **A2. Draft saving:** 1. Before submitting, professor chooses "Save as Draft". 2. System saves a draft that is not yet visible to coordinators or students. 3. Professor can later edit the draft and submit it.

Table II.10: Use Case Specification: Secretariat Verifies Payment and Finalizes Protocol

Use Case ID	UC-10
Use Case Name	Secretariat Verifies Payment and Finalizes Protocol
Primary Actor	Secretariat
Secondary Actors	Company Representative, Student, System
Preconditions	- A student has been assigned to a proposal that requires a protocol or payment. - Required protocol documents and/or payment information are available (uploaded or received externally). - Secretariat staff is authenticated with appropriate permissions.
Postconditions	- Payment status and protocol status are updated (e.g., "Verified", "Signed"). - The associated internship/dissertation is marked as "Active" or "Authorized". - Relevant parties can see the updated status.
Main Flow	1. Secretariat logs in and opens the "Protocols / Payments" section. 2. System shows a list of pending cases (assigned proposals requiring verification). 3. Secretariat selects a specific case. 4. System shows student, company, proposal details, and any uploaded documents or payment records. 5. Secretariat verifies that all required documents are signed and any required payments are confirmed. 6. Secretariat updates the status to "Verified / Finalized" and saves. 7. System updates the associated assignment to "Active" and notifies the company and student.

Alternative Flows	**A1. Missing or invalid documentation:** 1. At step 5, Secretariat detects missing signatures or incorrect information. 2. Secretariat sets the status to "Incomplete" and records a comment. 3. System notifies the company and/or student with the requested corrections. **A2. Payment not received:** 1. Payment is not yet confirmed. 2. Secretariat records the issue and leaves the status as "Pending Payment". 3. No activation occurs until payment is confirmed.

Table II.11: Use Case Specification: Edition Coordinator Publishes Call for Proposals

Use Case ID	UC-11
Use Case Name	Edition Coordinator Publishes Call for Proposals
Primary Actor	Edition Coordinator
Secondary Actors	System, Company Representatives
Preconditions	- At least one edition exists in "Configured" state with defined proposal submission dates. - Coordinator is authenticated and has edition management rights. - Company contacts exist or can be targeted by the call.
Postconditions	- The edition's call for proposals is marked as "Published". - Companies can see the call when logging in and can submit proposals for that edition. - Notification emails are sent to targeted companies.
Main Flow	1. Coordinator opens the "Editions" view and selects a configured edition. 2. System shows current configuration and call status. 3. Coordinator reviews details (dates, constraints) and chooses "Publish Call for Proposals". 4. System requests confirmation and optionally allows editing the email template. 5. Coordinator confirms publication. 6. System changes the edition state to "Call Published / Open for Proposals". 7. System sends notification emails to configured company recipients and updates the dashboard.
Alternative Flows	**A1. Misconfigured edition:** 1. At step 3, system detects missing critical parameters (e.g., no submission window). 2. System blocks publication and presents a list of missing items. 3. Coordinator fixes the configuration, then retries. **A2. Email sending issues:** 1. At step 7, some emails fail to send. 2. System logs failures and informs the coordinator, offering a way to export the list of failed addresses.

Table II.12: Use Case Specification: Administrator Performs Superuser Access for Student Support

Use Case ID	UC-12
Use Case Name	Administrator Performs Superuser Access for Student Support
Primary Actor	System Administrator
Secondary Actors	Student, Coordinator, Company Representative, System
Preconditions	- Administrator is authenticated with elevated permissions. - A support request has been raised (e.g., user cannot access their account, data inconsistency, wrong role assignment). - Superuser actions are enabled and auditable.
Postconditions	- The specific problem (access, data correction, role fix) is resolved. - All actions executed under superuser mode are logged with timestamp, actor, and justification.
Main Flow	1. Administrator opens the "Support / Superuser Access" area. 2. System lists open support tickets or allows searching by user, edition, or proposal. 3. Administrator selects a case and views context (user roles, assignments, logs). 4. Based on the case the administrator logs in the Superuser with the other user's login. 5. Administrators troubleshoots the problems. 6. Logs out of the accounts being accessed. [A2]
Alternative Flows	**A1. Action not allowed by policy:** 1. The requested operation violates locked-data policy. 2. System blocks the action and explains that such changes require a different process. 3. Administrator records this outcome in the ticket. **A2. Impersonation session ended:** 1. During impersonation, administrator finishes troubleshooting. 2. Administrator exits superuser session explicitly. 3. System returns to the administrator's own account and logs the full session.